

RECURSION

Recursion is a technique in which a function calls itself to solve a problem by breaking it down into smaller subproblems.

1. HOW IT WORKS

- **Base Case** (Stopping Condition): The condition under which the recursion stops.
- **Recursive Case**: The part where the function calls itself with a smaller input.

2. EXAMPLE: FACTORIAL (n!)

Factorial of n ($n!$) = $1 \times 2 \times 3 \times \dots \times n$

Recursive definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

Code (Python):

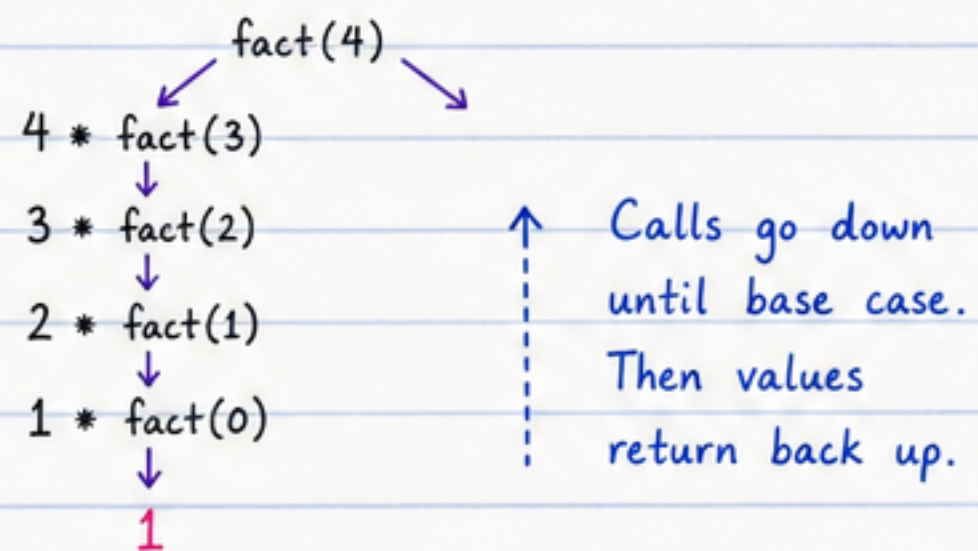
```
def fact(n):  
    if n == 0:  
        return 1 # Base case  
    else:  
        return n * fact(n-1) # Recursive case
```

3. EXAMPLE CALL

fact(4)
= 4 * fact(3)
= 4 * (3 * fact(2))
= 4 * (3 * (2 * fact(1)))
= 4 * (3 * (2 * fact(0)))
= 4 * (3 * (2 * 1))
= 24

← Base case reached

4. RECURSION TREE (fact(4))



5. KEY POINTS

- Every recursive function must have a base case.
- Each recursive call should move toward the base case (smaller input).
- Recursion uses more memory (call stack).
- Can often be converted to iteration (loops).

Time Complexity (example):

$$T(n) = T(n-1) + O(1)$$

$$\Rightarrow T(n) = O(n)$$

≡ DYNAMIC PROGRAMMING ≡

1. WHAT IS IT?

- A technique for solving problems by breaking them into overlapping subproblems and solving each subproblem only once.
- Combines the ideas of recursion and memoization (remembering answers).
- Used to optimize problems with **optimal substructure** and **overlapping subproblems**.

2. WHEN TO USE?

- The problem can be divided into subproblems.
- The same subproblems are solved multiple times in a naive solution.
- The problem has optimal substructure (optimal solution can be built from optimal solutions of subproblems).

3. KEY IDEAS

- **State**: Define a state that represents a subproblem.
- **Transition**: Define how to compute the answer for a state using smaller states.
- **Base Case**: Define the simplest states (base cases).
- **Memoization / Tabulation**: Store results of solved states to avoid recomputation.

4. APPROACHES

Top-Down (Memoization)

- Use recursion.
- Store results in a table/map.
- Time $\approx O(\text{number of states})$
- Space $\approx O(\text{number of states}) + \text{recursion stack}$

Bottom-Up (Tabulation)

- Build a table iteratively.
- Solve smaller states first.
- Time $\approx O(\text{number of states})$
- Space $\approx O(\text{number of states})$

5. GENERAL TEMPLATE

```
dp(state):  
    if state is base case:  
        return base value  
    if state already computed:  
        return memo[state]  
    res = compute using transitions  
    memo[state] = res  
    return res
```

```
for each state in increasing order:  
    if state is base case:  
        dp[state] = base value  
    else:  
        dp[state] = compute using transitions  
answer = dp[target_state]
```

Remember:

Define the state clearly — half the problem is solved!



6. EXAMPLES

- Fibonacci Numbers

$$dp[n] = dp[n-1] + dp[n-2]$$

Base: $dp[0]=0, dp[1]=1$

- 0/1 Knapsack

$$dp[i][w] = \max(dp[i-1][w], \text{value}_i + dp[i-1][w - \text{weight}_i])$$

Base: $i=0$ or $w=0$

- Longest Common Subsequence (LCS)

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & \text{if } s_1[i-1] == s_2[j-1] \\ \max(dp[i-1][j], dp[i][j-1]), & \text{otherwise} \end{cases}$$

Base: $i=0$ or $j=0$

★ Dynamic Programming turns exponential solutions into polynomial ones by reusing work. 😊